DuocUC® | ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

# Testing FrontEnd

# Proyecto “Locuras Dulces”


Asignatura: FullStack II

Sección: 003V

Docente: Viviana Poblete López

Integrantes: Vicente Quiroz

Paula Rojas

## 1. Introducción y propósito

El presente documento expone el proceso de implementación de pruebas en el proyecto Locuras Dulces, desarrollado con React y Vite. El objetivo principal del testing es garantizar que los componentes y páginas del sistema funcionen de acuerdo con lo esperado, permitiendo detectar errores de manera temprana y asegurar la calidad del código antes de su despliegue.

Asimismo, este trabajo busca dar cumplimiento a los requisitos mínimos establecidos en la rúbrica académica, aplicando buenas prácticas de testing automatizado mediante herramientas modernas como Vitest y React Testing Library. Con ello se fortalece la confiabilidad del proyecto y se sientan las bases para un mantenimiento y evolución más seguros en el futuro.

## 2. Herramientas y configuración

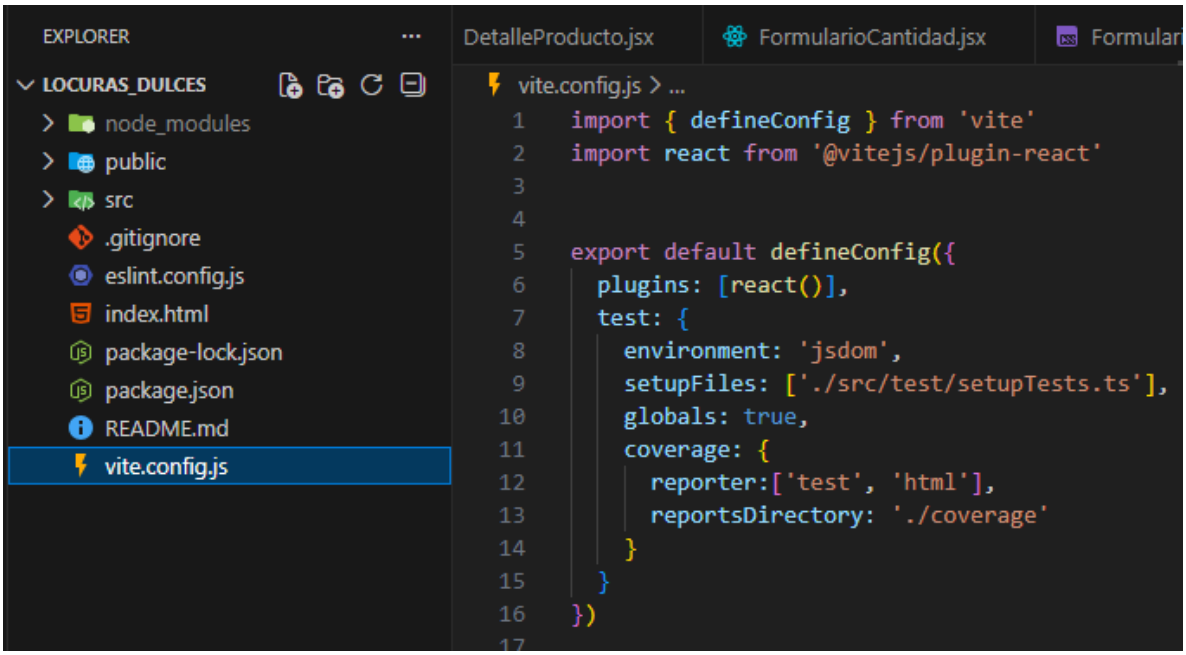
Para el entorno de testing se utilizaron las siguientes herramientas:

- **Vitest**: motor de pruebas compatible con Vite.
- **React Testing Library**: para interactuar con componentes como lo haría un usuario.
- **Jest DOM**: para extender los matchers de aserción.
- **user-event**: para simular acciones del usuario.
- **jsdom**: para simular el DOM en entorno de pruebas.
- **@vitest/ui**: interfaz gráfica para visualizar los resultados.

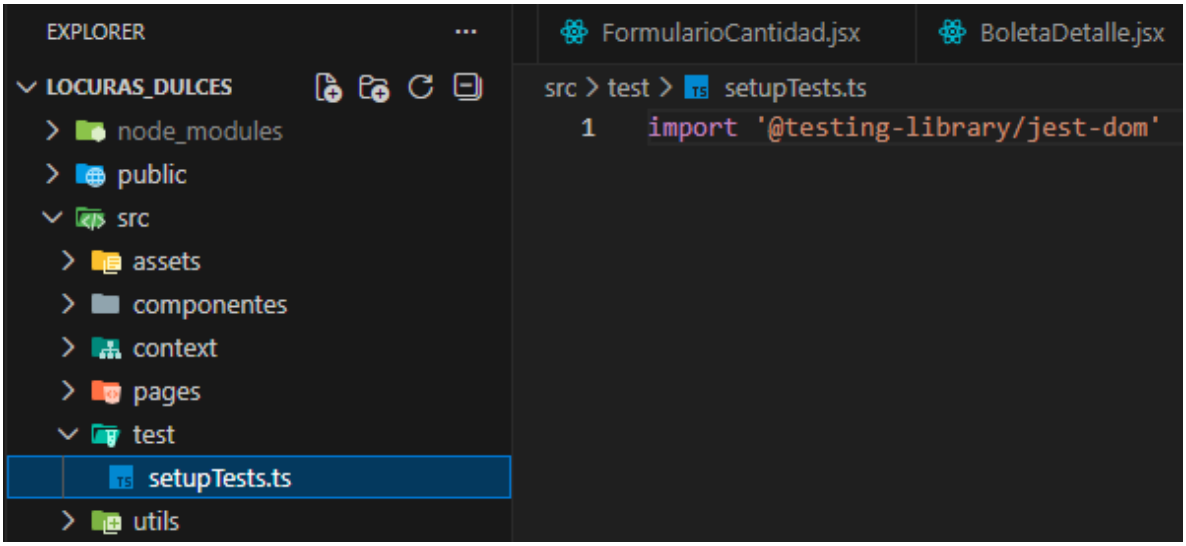
## 3. Instalación de dependencias

npm install --save-dev vitest @testing-library/react @testing-library/jest-dom @testing-library/user-event jsdom @vitest/ui

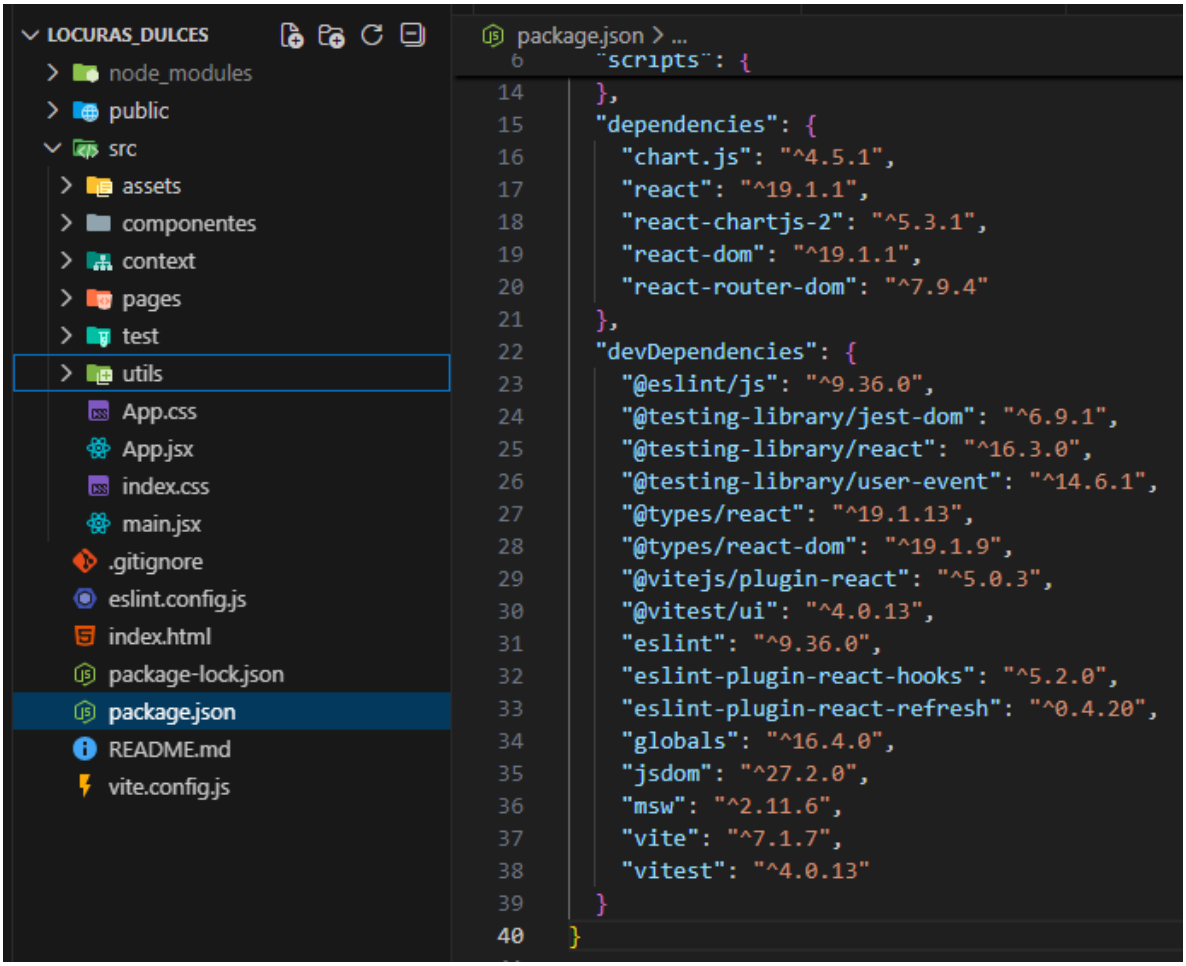
1. Configuración en vite.config.js




2. Archivo setupTests.js



### 3. Scripts en package.json




## 4. Plan de testing

### 1. Componentes/pages testeados

Se seleccionaron 5 componentes/pages clave del proyecto:

1. Navbar.jsx — navegación y roles de usuario
2. CarritoContext.jsx — lógica de carrito y persistencia
3. FormularioCompra.jsx — validación de compra y envío de boleta
4. BoletaDetalle.jsx — renderizado de boleta y productos
5. CarritoPage.jsx — interacción con el carrito y navegación

Ejemplos de código

2. Navbar.test.jsx

```
Navbar.test.jsx ✕   DetalleProducto.jsx   FormularioCantidad.jsx   BoletaDetalle.jsx   Bolet
src > componentes > Navbar > Navbar.test.jsx > ...
15  describe('Navbar', () => {
18    })
19
20    it('muestra saludo de visitante por defecto', () => {
21      render(
22        <MemoryRouter>
23          <Navbar />
24        </MemoryRouter>
25      )
26      expect(screen.getByText(/bienvenido, visitante/i)).toBeInTheDocument()
27      expect(screen.getByRole('link', { name: /home/i })).toHaveAttribute('href', '/')
28    })
```

3. CarritoContext.test.jsx

```
src > context > CarritoContext.test.jsx > ...
24  describe('CarritoContext', () => {
29    it('agrega y acumula cantidades del mismo producto', async () => {
30      const user = userEvent.setup()
31      render(
32        <CarritoProvider>
33          <Dummy />
34        </CarritoProvider>
35      )
36      await user.click(screen.getByText(/agregar a$/i))
37      await user.click(screen.getByText(/agregar a x2/i))
38      expect(screen.getByTestId('count')).toHaveTextContent('3')
39    })
```

## 4. FormularioCompra.test.jsx

src > pages > FormularioCompra > FormularioCompra.test.jsx > ...

```
16  describe('FormularioCompra', () => {
40    it('envía payload correcto con usuario y carrito', async () => {
41      const user = userEvent.setup()
42      localStorage.setItem('usuarioActivo', JSON.stringify({ id: 7, nombre: 'Cliente' }))
43      localStorage.setItem('carrito', JSON.stringify([{ id: 5, nombre: 'Producto', precio: 1700, cantidad: 1 }]))
44
45      global.fetch.mockResolvedValueOnce({
46        ok: true,
47        json: async () => ({ idBoleta: 123, total: 1700 })
48      })
49
50      render(
51        <MemoryRouter initialEntries={['/formulario-compra']}>
52          <CarritoProvider>
53            <FormularioCompra />
54          </CarritoProvider>
55        </MemoryRouter>
56      )
57
58      await user.click(screen.getByRole('button', { name: /pagar ahora/i }))
59      expect(global.fetch).toHaveBeenCalledWith(
60        'http://localhost:8080/api/boletas',
61        expect.objectContaining({
62          method: 'POST',
63          headers: { 'Content-Type': 'application/json' },
64          body: expect.stringContaining('"idUsuario":7')
65        })
66      )
```

## 5. BoletaDetalle.test.jsx

pages > Boleta > BoletaDetalle.test.jsx > ...

```
describe('BoletaDetalle', () => {
  it('muestra datos de boleta y productos', async () => {

    // Verifica datos generales
    expect(screen.getByText(/débito/i)).toBeInTheDocument()
    expect(screen.getByRole('heading', { name: /total: \$1\.700/i })).toBeInTheDocument()

    // Verifica producto
    expect(screen.getByText(/torta selva negra/i)).toBeInTheDocument()

    // Busca solo las celdas de tabla con $1.700 (precio unitario y subtotal)
    const preciosTabla = screen.getAllByRole('cell', { name: /\$1\.700/ })
    expect(preciosTabla).toHaveLength(2)
  })

  it('muestra mensaje de error si la boleta no existe', async () => {
    global.fetch.mockResolvedValueOnce({ ok: false })

    render(
      <MemoryRouter initialEntries={['/boleta/99']}>
        <Routes>
          <Route path="/boleta/:id" element={<BoletaDetalle />} />
        </Routes>
      </MemoryRouter>
    )

    expect(await screen.findByText(/boleta no encontrada/i)).toBeInTheDocument()
  })
```

## 6. CarritoPage.test.jsx

```
describe('CarritoPage', () => {

  it('navega a formulario de compra al finalizar compra', async () => {
    const user = userEvent.setup()
    localStorage.setItem(
      'carrito',
      JSON.stringify([{ id: 1, nombre: 'Brownie', precio: 1000, cantidad: 1 }])
    )

    render(
      <MemoryRouter initialEntries={['/carrito']}>
        <Routes>
          <Route
            path="/carrito"
            element={
              <CarritoProvider>
                <CarritoPage />
              </CarritoProvider>
            }
          />
          <Route path="/formulario-compra" element={<h1>FormularioCompra</h1>} />
        </Routes>
      </MemoryRouter>
    )

    await user.click(screen.getByRole('button', { name: /finalizar compra/i }))
    // Verificamos por rol heading para mayor robustez
    expect(await screen.findByRole('heading', { name: /FormularioCompra/i })).toBeInTheDocument()
  })
```

# 5.Resultados de los testing

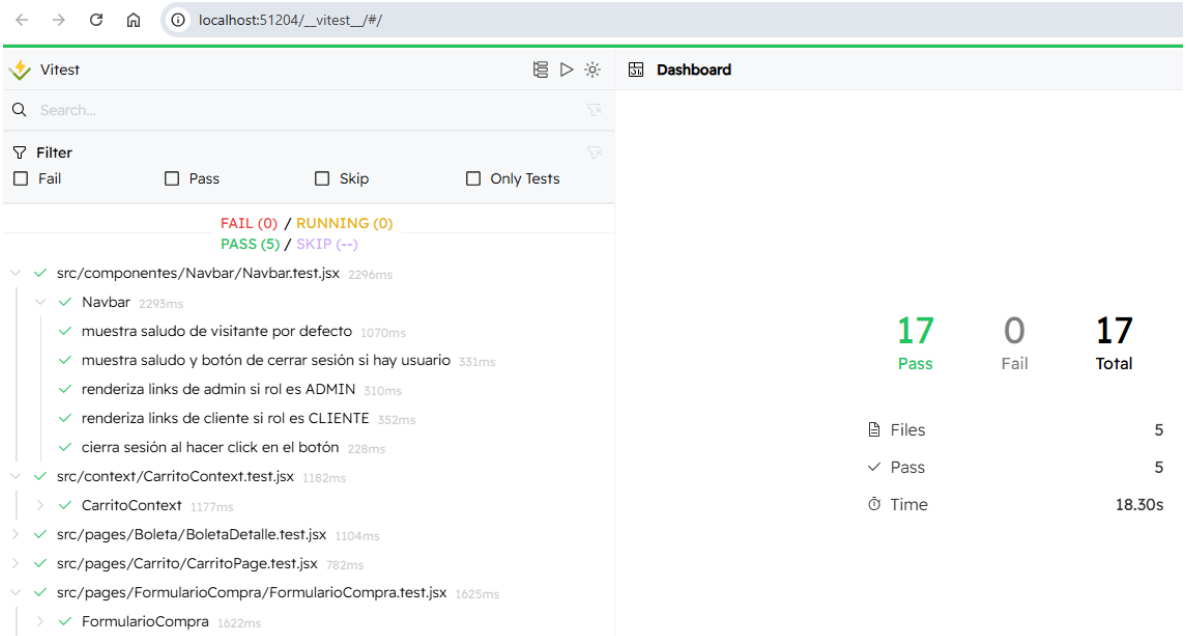
## 1. Consola: ejecución con **npm run test**

```
PS C:\Users\Vicente\Documents\Cursos y Capacitaciones\DUOC\2DO SEMESTRE 2025\FullStack2\Locuras_Dulces> npm run test

> frontend@0.0.0 test
> vitest


 DEV  v4.0.13 C:/Users/Vicente/Documents/Cursos y Capacitaciones/DUOC/2DO SEMESTRE 2025/FullStack2/Locuras_Dulces

stdout | src/pages/FormularioCompra/FormularioCompra.test.jsx > FormularioCompra > envía payload correcto con usuario y carrito
Payload boleta (enviando): {
  idUsuario: 7,
  medioPago: 'Débito',
  direccion: {
    calle: null,
    numero: null,
    comuna: null,
    region: null,
    referencia: null
  },
  detalles: [ { idProducto: 5, cantidad: 1 } ]
}

stdout | src/pages/FormularioCompra/FormularioCompra.test.jsx > FormularioCompra > envía payload correcto con usuario y carrito
Boleta recibida: { idBoleta: 123, total: 1700 }

stdout | src/pages/FormularioCompra/FormularioCompra.test.jsx > FormularioCompra > redirige a compra-fallida si fetch falla
Payload boleta (enviando): {
  idUsuario: 7,
  medioPago: 'Débito',
  direccion: {
    calle: null,
    numero: null,
```

```
    calle: null,
    numero: null,
    comuna: null,
    region: null,
    referencia: null
  },
  detalles: [ { idProducto: 5, cantidad: 1 } ]
}

stderr | src/pages/FormularioCompra/FormularioCompra.test.jsx > FormularioCompra > redirige a compra-fallida si fetch falla
Error response: undefined Error
Error: Error al generar boleta
    at handleConfirmarCompra (C:/Users/Vicente/Documents/Cursos y Capacitaciones/DUOC/2DO SEMESTRE 2025/FullStack2/Locuras_Du
arioCompra/FormularioCompra.jsx:104:15)

 ✓ src/pages/FormularioCompra/FormularioCompra.test.jsx (3 tests) 1050ms
     ✓ redirige a login si no hay usuario  707ms
 ✓ src/componentes/Navbar/Navbar.test.jsx (5 tests) 1507ms
     ✓ muestra saludo de visitante por defecto  696ms
 ✓ src/context/CarritoContext.test.jsx (3 tests) 972ms
     ✓ agrega y acumula cantidades del mismo producto  606ms
 ✓ src/pages/Boleta/BoletaDetalle.test.jsx (2 tests) 866ms
     ✓ muestra datos de boleta y productos  850ms
 ✓ src/pages/Carrito/CarritoPage.test.jsx (4 tests) 805ms
     ✓ vacía el carrito al hacer click en el botón  547ms

 Test Files  5 passed (5)
      Tests  17 passed (17)
   Start at  15:28:21
   Duration  13.23s (transform 760ms, setup 1.94s, collect 3.86s, tests 5.20s, environment 9.24s, prepare 209ms)

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

Resultado en consola — 5 archivos, 17 tests aprobados

## 2. Interfaz gráfica: ejecución con **npm run test:ui**




Interfaz gráfica — Dashboard de Vitest con todos los tests aprobados

## 6.Conclusión

El proceso de testing realizado en Locuras Dulces permitió comprobar de manera sistemática el correcto funcionamiento de los principales componentes y páginas del proyecto. A través de pruebas unitarias e integraciones se validaron aspectos esenciales como la interacción del usuario en la navegación, la gestión del carrito de compras, la confirmación de órdenes y la visualización de boletas.

La ejecución de los tests en consola y en la interfaz gráfica de Vitest demostró resultados consistentes, con todos los casos superados satisfactoriamente. Esto confirma que las funcionalidades críticas del sistema responden de acuerdo a lo esperado y que la aplicación se encuentra en un estado estable.

Además, la configuración del entorno de pruebas quedó preparada para futuras iteraciones, lo que facilita la incorporación de nuevos casos de prueba y asegura que el proyecto pueda mantenerse y evolucionar sin comprometer su calidad. En conjunto, este trabajo de testing aporta confianza en la solución desarrollada y evidencia el cumplimiento de los criterios establecidos en la rúbrica académica.